



Codi Embegut: HTML + PHP

R2 • Sessió 1

Una introducció pràctica a la convivència entre HTML i PHP en un mateix fitxer: com funciona, per a què serveix, i com fer-ho bé des del principi.

Què entenem per codi embegut?

La idea fonamental

Parlem de **codi embegut** quan dins d'un document HTML apareixen fragments de PHP. Els dos llenguatges conviuen en el mateix fitxer, però cadascú fa la seva feina.

HTML estructura la pàgina. PHP genera contingut dinàmic.

Per a què serveix?

→ **Mostrar valors**

Variables, resultats de càlculs o dades de l'usuari.

→ **Mostrar errors**

Missatges de validació dins del formulari.

→ **Condicions visuals**

Decidir si un bloc de la pàgina apareix o no.

→ **Generar contingut**

Recórrer arrays i crear llistes dinàmiques.

El Primer Exemple: Mínim i Clar

El millor punt de partida és el més senzill possible. Observa com PHP genera el text que apareix dins de l'HTML:

```
<?php
$titol = "Benvinguda";
?>
<!DOCTYPE html>
<html>
  <head><title>Exemple mínim</title></head>
  <body>
    <h1><?= htmlspecialchars($titol) ?></h1>
    <p>Açò és una pàgina generada amb PHP.</p>
  </body>
</html>
```

 L'HTML **estructura** la pàgina. PHP **genera el contingut**. Aquesta divisió de responsabilitats és la clau per entendre el codi embegut.

Variable Embeguda dins del HTML

Un dels usos més habituals: preparar una variable en PHP i inserir-la dins del document HTML de manera segura.

```
<?php
$nom = "Maria";
?>
<p>Benvinguda, <?= htmlspecialchars($nom) ?>!</p>
```

Pas 1: Preparar

La variable `$nom` es defineix al bloc PHP **abans** del HTML.

Pas 2: Inserir

S'usa `<?= ... ?>` per imprimir el valor dins de l'etiqueta HTML.

Pas 3: Escapar

`htmlspecialchars()` converteix caràcters especials i evita injeccions HTML.

Mostrar un Missatge Condicional

El codi

```
<?php
$hi_ha_error = true;
?>

<?php if ($hi_ha_error): ?>
  <p class="error">
    Hi ha un error en el formulari.
  </p>
<?php endif; ?>
```

Què aporta aquest exemple?

PHP decideix si el bloc apareix o no. L'HTML continua sent recognoscible i la lectura del codi és neta.

PHP controla la visibilitat

El condicional `if/endif` envolta el bloc HTML.

HTML es manté llegible

Les etiquetes no queden amagades dins de cadenes de text PHP.

- ❏ Idea clau: el codi embegut és molt útil per **mostrar o amagar parts de la pàgina** segons una condició.

Errors de Validació i Recuperació de Dades

Dos patrons fonamentals que trobareu constantment en formularis reals:

Mostrar error de validació

```
<?php
$errors = ['nom' => 'El nom és obligatori.'];
?>

<label>Nom</label>
<input type="text" name="nom">
<?php if (isset($errors['nom'])): ?>
  <p class="error">
    <?= htmlspecialchars($errors['nom']) ?>
  </p>
<?php endif; ?>
```

Recuperar el valor escrit

```
<input
  type="text"
  name="nom"
  value="<?= htmlspecialchars(
    $_POST['nom'] ?? "
  ) ?>"
>
```

El camp recupera el valor escrit per l'usuari. Així no cal tornar a escriure-ho tot si hi ha un error. HTML i PHP col·laboren en una mateixa resposta.

Llistes Generades des d'un Array

PHP pot recórrer un array i generar automàticament tants elements HTML com siguin necessaris:

```
<?php
$fruites = ['Poma', 'Pera', 'Taronja'];
?>

<ul>
  <?php foreach ($fruites as $fruita): ?>
    <li><?= htmlspecialchars($fruita) ?></li>
  <?php endforeach; ?>
</ul>
```



Dades separades

L'array es defineix al bloc PHP inicial, mantenint les dades separades de la presentació.



Generació automàtica

`foreach` crea tants `<li>` com elements hi hagi, sense repetir codi HTML.



Eixida segura

`htmlspecialchars()` s'aplica a cada element per garantir la seguretat.

Un Mal Exemple: Què Evitar

✗ Codi poc recomanable

```
<?php
if ($_POST['nom'] != "") {
    echo "<p>Hola, " . $_POST['nom'] . "</p>";
} else {
    echo "<p style='color:red'>Error</p>";
}
?>
```

Per què és problemàtic?

Accés directe a \$_POST

Sense comprovar si la clau existeix, pot generar advertències.

Sense escapar l'eixida

Imprimeix directament valors de l'usuari: risc d'injecció HTML/XSS.

Mescla caòtica

L'HTML dins de cadenes PHP és difícil de llegir i de modificar.

Errors pobres

El missatge d'error no és descriptiu ni útil per a l'usuari.

⊗ No tot codi embegut està igual de ben resolt. La qüestió és fer-lo **llegible, segur i fàcil d'explicar**.

Bones Pràctiques Mínimes

01

Preparar dades abans del HTML

Tot el PHP de lògica va al principi del fitxer, abans de qualsevol etiqueta HTML.

02

Fragments PHP curts i concrets

Dins del HTML, els blocs PHP han de ser breus: una condició, un echo, un bucle.

03

Escapar sempre l'eixida

Usa `htmlspecialchars()` en tot valor que vingui de l'usuari o de dades externes.

04

Errors clars i descriptius

Els missatges d'error han d'ajudar l'usuari a entendre què ha passat i com corregir-ho.

05

Mantenir l'HTML recognoscible

Si el document és difícil de llegir com a HTML, és senyal que hi ha massa lògica mesclada.

Quan és Útil i Quan Comença a Ser un Problema

✓ És útil quan...

- Estem aprenent la base del funcionament servidor
- El flux de la pàgina és simple i directe
- Volem entendre com es genera la resposta HTTP
- Mostrem valors, errors o llistes curtes

⚠ Comença a ser un problema quan...

- Tota la lògica del projecte viu dins de la pàgina
- Hi ha moltes condicions i bucles mesclats amb HTML
- Costa distingir què és HTML i què és PHP
- El fitxer creix i es fa difícil de mantenir

⚠ Aquesta forma de treballar és molt bona per **començar**, però té límits quan el projecte creix. Més endavant aprendrem a separar responsabilitats.

Resum: Què Ha d'Entendre l'Alumnat



Codi embegut

Què és i com conviuen HTML i PHP en un mateix fitxer.



Inserir valors

Com mostrar una variable PHP dins d'una etiqueta HTML de forma segura.



Errors condicionals

Com mostrar missatges d'error sols quan es compleix una condició.



Llistes dinàmiques

Com generar elements HTML repetits a partir d'un array amb foreach.



Recuperar dades

Com repoblar un formulari amb el valor que l'usuari havia introduït.



Bona vs. mala mescla

Reconèixer la diferència entre una integració clara i una de desordenada.

No es tracta només de "barrejar PHP dins del HTML", sinó de fer-ho d'una manera **clara, útil i defensable**.